Процедурная генерация

Процедурная генерация

Зачем?

Допустим, вы выбрали рогалик. Или любой другой жанр, где уровни должны отличаться друг от друга при каждом прохождении. У вас 20, 50, 100 уровней. Вы правда хотите делать каждый вручную?

Можно, конечно. Но это огромное количество работы, и при повторном прохождении игрок увидит те же самые уровни. Реиграбельность стремится к нулю.

А что, если уровни будут создаваться программой? Каждый раз разные, но при этом играбельные. Это и есть **процедурная генерация** - создание игрового контента алгоритмически, а не вручную.

> Процедурная генерация - не замена дизайну уровней. Это инструмент. Сгенерированный уровень все равно должен подчиняться каким-то правилам, иначе он будет неиграбельным.

Что можно генерировать?

В целом - почти все:

- **Уровни и подземелья**: комнаты, коридоры, пещеры.
- **Ландшафт**: горы, равнины, реки, биомы.
- **Предметы**: случайные характеристики оружия, лута.
- **Враги**: расстановка, параметры, поведение.
- **Сюжет и квесты**: хотя это уже довольно сложная тема.

При чём тут тайлы

Вспомним, что в разделе про уровни мы разбивали ландшафт на тайлы и хранили их в виде матрицы. Каждый тайл - это ячейка матрицы с какими-то характеристиками.

Т.е. у нас уже есть структура данных для хранения уровня. Генерация - это, грубо говоря, **заполнение этой матрицы значениями**. Стена, пол, дверь, вода - все это разные значения ячеек. Например, `0` - пол, `1` - стена.

Вопрос в том, как именно заполнять.

Наивный подход

Самый простой вариант: пройтись по каждой ячейке и случайным образом решить, стена это или пол. Примерно так:

Python C#

```python
import random

def generate_random_map(width, height, wall_chance=0.5):
    grid = []
    for y in range(height):
        row = []
        for x in range(width):
            if random.random() < wall_chance:
                row.append(1)  # wall
            else:
                row.append(0)  # floor
        grid.append(row)
    return grid

level = generate_random_map(20, 15, 0.4)
```

Результат? Бред. Буквально. Каждая ячейка не зависит от соседних, поэтому мы получаем хаотичную кучу стен и пола без какой-либо структуры. Нет комнат, нет коридоров, нет проходимых путей. Играть невозможно.

Ок. Рандом сам по себе не работает. Нам нужна какая-то **структура**, какие-то **правила**, по которым генерация будет происходить. И тут есть несколько подходов, каждый из которых решает задачу по-своему.

Подходы к генерации уровней

Случайное блуждание (Random Walk)

Представьте, что у вас есть "строитель", который стоит в центре матрицы, заполненной стенами. Он случайным образом выбирает направление, делает шаг и "прорезает" пол на своём пути. Повторяем тысячу раз - получаем систему извилистых тоннелей.

Результат напоминает органические пещеры - никаких прямоугольных комнат, всё плавное и хаотичное. Алгоритм элементарный, буквально 15-20 строк кода.

Подробнее: Случайное блуждание

BSP (Binary Space Partitioning)

BSP уже упоминался в контексте коллизий и разбиения пространства. Здесь идея в следующем: берём всю область уровня и рекурсивно делим её пополам - вертикально или горизонтально. Повторяем,

пока части не станут достаточно маленькими. В каждой такой части размещаем комнату, а затем соединяем соседние комнаты коридорами.

Результат - классические подземелья с комнатами и коридорами, как в Binding of Isaac или Enter the Gungeon. Структура чёткая и предсказуемая.

Подробнее: BSP-генерация

Клеточные автоматы (Cellular Automata)

Помните наивный подход со случайным заполнением? А что, если мы возьмём его результат и "сгладим"? Идея: заполняем матрицу случайно (скажем, 45% стен), а затем несколько раз проходим по ней с простым правилом - если у ячейки много соседей-стен, она тоже становится стеной, иначе - полом.

После нескольких таких проходов хаос превращается в гладкие, естественно выглядящие пещеры. Алгоритм очень компактный, а результат визуально впечатляет.

Подробнее: Клеточные автоматы

Шум Перлина (Perlin Noise)

Предыдущие подходы работают с дискретными значениями - стена или пол. Шум Перлина - это другая история. Он генерирует плавные, непрерывные значения (от 0.0 до 1.0), из которых можно создавать ландшафт: значение ниже 0.3 - вода, от 0.3 до 0.6 - трава, выше 0.6 - горы.

Это подходит для открытых миров, карт высот, распределения биомов. Но не для подземелий с комнатами и коридорами - для этого лучше использовать подходы выше.

Подробнее: Шум Перлина

Wave Function Collapse (WFC)

А что, если тайлы сами будут решать, кто может стоять рядом? Задаём правила соседства: вода может граничить с песком, песок - с травой, трава - с лесом. Заполняем матрицу, начиная с ячейки с минимальной энтропией (наименьшим числом вариантов), и распространяем ограничения на соседей.

Результат - локально согласованный мир, где каждый тайл логично сочетается с соседями. Подход мощнее случайного заполнения, но требует ручного описания правил.

Подробнее: Wave Function Collapse

L-системы

Все предыдущие подходы заполняют сетку. Но как сгенерировать дерево, куст, коралл - ветвящуюся структуру? L-система - это формальная грамматика: аксиома + правила замены. На каждой итерации строка переписывается, затем интерпретируется как команды для "черепашки" - и из нескольких символов вырастает фрактальное растение.

Это не метод генерации уровней, а инструмент для **визуальной** процедурной генерации: деревья, кусты, трава, узоры.

Подробнее: L-системы

Seed - зерно генерации

Все перечисленные алгоритмы используют случайные числа. Но случайные числа в программировании - не совсем случайные. Они генерируются на основе начального значения - **seed** (зерно). Если мы зафиксируем seed, то каждый запуск генерации даст одинаковый результат.

Зачем? Во-первых, для дебага - вы можете воспроизвести конкретный уровень, на котором что-то сломалось. Во-вторых, для обмена - игроки могут делиться интересными уровнями, передавая seed. Minecraft, к примеру, именно так и работает.

Python C#

```python
import random

seed = 42
random.seed(seed)
level = generate_random_map(20, 15, 0.4)
# every run with seed=42 produces the same level
```

Валидация: можно ли пройти уровень?

Ок. Мы сгенерировали уровень. Комнаты, пещеры, коридоры - выглядит красиво. Но есть ли путь от точки старта до выхода? Может, генератор случайно замуровал одну из комнат. Или две системы пещер не соединены между собой. А в одной спавниться игрок, а в другой находится выход с уровня. Хм.

Так что, наверное, стоит проверить, что уровень проходим?

Вспомним, что наша матрица тайлов - это, по сути, граф. А задача "можно ли добраться из A в B" - классическая задача на графе.

Самый простой способ - **заливка** (flood fill). Идея такая: начинаем со стартовой точки, "заливаем" все доступные (проходимые) ячейки, как краску по полу. Если выход оказался залит - уровень проходим. Если нет - генерируем заново или прокладываем дополнительный коридор. По сути, BFS.

Более того, заливка может показать, сколько **отдельных регионов** существует на уровне. Это полезно, например, если вы хотите разместить ключ в одном регионе, а дверь - в другом, заставив игрока найти способ добраться.

Валидация - не обязательный шаг. Если ваш алгоритм **гарантирует** связность (как, например, BSP, где все комнаты соединяются коридорами), то проверять ничего не нужно (гарантию еще надо математически доказать). Но если вы используете клеточные автоматы или случайное блуждание, где связность не гарантирована, проверка точно не помешает.

Что	Где подробнее
Как сделать "честный" рандом, seed-менеджмент, PRD	Случайность и вероятность
Поиск пути по сгенерированным уровням	Уровни и поиск путей
Коллизии в процедурных уровнях	Коллизии
Эволюция и оптимизация уровней	Генетические алгоритмы
Вероятностные переходы для генерации текста, имён	Цепи Маркова

Powered by Wiki.js